CIS*6530 · Lecture 1 exercises: security data wrangling

Estimated effort: 6 to 8 hours. **Not marked, no due date.** Do finish Exercise 5 before Lecture 2, which starts from its output. Data is in `data/lecture1/` (see `docs/DATA_DICTIONARY.md`). Starter files are in `starter/lecture1/`.

Rules for every exercise

1. **Never modify raw data.** Write everything to `out/`.
2. **Reproducible from a clean clone.** `make -C starter/lecture1 all` (or your own script) must regenerate every output from the raw files.
3. **State your assumptions** in `NOTES.md`, including time zones, how you treated bad rows, and every threshold you chose.
4. **Every number in your notes must be traceable** to a command or cell.
5. Use only this data pack. Do not upload it anywhere.
6. Check yourself with `python tools/check_lecture1.py --data data/lecture1 --out out`. It checks structure and a few sanity facts. It does **not** judge correctness: that is your job.

Exercise 1: jq: normalise alerts (Lecture 1, jq block)

Input: `eve.json` **Tool:** jq (post-processing allowed) **Output:** `out/alerts.csv`, `out/bucket_counts.csv`, section 1 of `NOTES.md`

Produce one row per alert event with this header, in this order (extra columns after these are allowed):

`ts,bucket,src,dst,port,signature,severity`

- `ts`: UTC epoch seconds (integer). `bucket`: `floor(ts/300)*300`.
- Only `event_type == "alert"`. Drop rows whose timestamp cannot be parsed, but **count them**.
- The sensor clock is not UTC. Work out its offset from the data, do not assume it. Verify with at least one event on each side of 8 March.
- The sensor sometimes retransmits an event. Decide what to do about duplicates and justify it.

Report in NOTES.md: (a) which time zone the sensor logs in and how you verified it; (b) how many alert events had unparseable timestamps; (c) how you handled duplicates; (d) alerts per 5-minute bucket (`bucket_counts.csv`, columns `bucket,n`) and the busiest UTC hour. Say what you think caused the busiest hour and how you would check.

Hints: jq's `fromdateiso8601` accepts only `YYYY-MM-DDTHH:MM:SSZ`. Use `capture` with a regex to split the offset, and remember jq will happily normalise impossible dates like 30 February unless you check.

Exercise 2: jq: flatten and enrich

Input: `eve.json`, `iocs.json` **Output:** `out/enriched.ndjson`, `out/ex2_memory.txt`

Write **one output line per input event** with every field flattened to a dotted key (e.g. `alert.severity`) plus two new fields: `ioc_hit` (true if `src_ip` or `dest_ip` is in `iocs.json`) and `src_private` (true if `src_ip` is RFC 1918). Use `--slurpfile` for the IOC list, a lookup object rather than an array scan, and `-n` with `inputs` so the file is streamed. Record peak memory: `/usr/bin/time -v` on Linux (`Maximum resident set size`), `/usr/bin/time -l` on macOS.

Report: peak memory, and one sentence on why streaming matters when the file is 100 times larger.

Exercise 3: awk: SSH report

Input: `auth.log` **Output:** `out/ssh_report.tsv`

Tab-separated, with this header: `src_ip failures distinct_users first_fail last_fail success_after_fail likely_compromise`

One row per source with **10 or more** `Failed password` lines. `success_after_fail` is `yes` if an `Accepted password` from the same address occurs *after* the 10th failure. `likely_compromise` is your judgement (`yes` / `no`). Read the raw log lines around each `yes` before you commit to it. Do it in one pass, in awk.

Report: how you defined `likely_compromise`; which sources look like password guessing against a few accounts and which look like username spraying (use `distinct_users`); and one source that is `success_after_fail = yes` but that you decided is *not* a compromise, with the evidence.

Exercise 4: awk + pandas: beaconing candidates

Input: `conn.log` **Output:** `out/deltas.tsv`, `out/beacons.csv`

1. **awk:** write `out/deltas.tsv` (header `pair delta_s orig_bytes`) with one row per connection after the first in each `orig_h>resp_h:resp_p` pair, where `delta_s` is the gap to the previous connection in that pair.
2. **pandas:** per pair with at least 50 connections compute `n`, `mean_gap`, `cv` (std/mean of gaps) and `size_cv` (coefficient of variation of `orig_bytes`). Rank candidates (a rank-average of low `cv`, low `size_cv` and few source hosts per destination works well) and write the **top 10** to `out/beacons.csv` with columns `pair,n,mean_gap,cv,size_cv,hosts,score`.

Report: for each of the ten, one line: *what would I check next?* (raw connections, destination reputation, is it on the IOC list, does a legitimate service explain it?). Identify at least two you believe are benign, and say why. Nothing in this list is a verdict.

Exercise 5: pandas: host-hour feature table (keystone: Lecture 2 uses it)

Input: everything **Output:** `out/features.parquet`, `out/audit.csv`

One row per `(src_ip, hour)` for every source that has at least one alert in that hour. `hour` is UTC, timezone-aware, floored to the hour. The key must be unique.

Column	Definition
`n_alerts, n_sigs, n_dst, n_ports`	alert count, distinct signatures, distinct destinations, distinct destination ports in the hour
`sev_min, sev_mean`	minimum and mean alert severity (1 is most severe)
`offhours`	1 if the hour is outside 08:00 to 18:00 America/Toronto Monday to Friday, else 0
`sig_rarity`	mean over the hour's alerts of `-log10(share of that signature)`. **Compute the share only from days before the alert's day**, or explain why you did not.

Add at least four optional features from: `dns_ent_max` (Shannon entropy of the first label of DNS names queried in the hour), `conn_n`, `conn_bytes_out`, `conn_n_dst`, `conn_max_pair` (connections to a single destination) from `conn.log`, `ioc_hit`, and `crit` from a time-aware lease join (`merge_asof`) followed by `assets.csv`. Document dtypes. Every feature must be computed from data available **within the hour**; the exceptions (`sig_rarity`, enrichment) must be justified in `NOTES.md`.

`out/audit.csv` has columns `check,finding,action` and must cover at least: unparseable timestamps, duplicates, missing values, key uniqueness, the share of events unmatched by lease or asset joins, and what you found in `assets.csv`.

Exercise 6: pandas: two baseline detectors and an analyst note

Input: `out/features.parquet` **Output:** `out/ranked.csv`, `out/top20_hosts.csv`, `out/ANALYST_NOTE.md` (one page)

1. **Robust z-score:** for a set of at least six count-like features, `log1p`-transform, compute the median and MAD **from the first 14 days only** (8 to 21 Feb), then score every host-hour as the maximum upward robust z. Floor the MAD so sparse features do not divide by zero, and say what floor you used.
2. **Rarity:** any defensible rarity score (`sig_rarity` alone is fine, richer is better).
3. `ranked.csv`: `src_ip,hour,rz_score,rz_rank,rarity_score,rarity_rank` (rank 1 = most anomalous).

4. `top20_hosts.csv`: top 20 sources for each method (max score over the source's hours) and how much they overlap.

Analyst note (one page): what in the top lists looks real, what looks benign and why, and your **estimate of the false positive rate** in the top 100 host-hours of each method, justified by reading a random sample of 20 of them. You will reuse these two detectors in Lecture 2 as the yardstick a model must beat.

Deliverables and quality bar

A git repository with all scripts, a `README` that reruns everything from the raw data, `out/` outputs, and `NOTES.md` + `ANALYST_NOTE.md`. Good work is reproducible, leaves the raw data untouched, states assumptions, and can point to the command behind every number.

Stretch: redo Exercise 5 in DuckDB. Benchmark jq, awk, pandas and DuckDB on a 1 GB extract (repeat runs, report medians, state hardware and cache state).